CUSTOM IC DESIGN, WITHOUT A CLOSED WORKFLOW

Give your design team another degree of freedom

Revolution EDA is an open, extensible environment for custom IC schematic, simulation, and layout workflows.

Executive briefing — current status and controlled adoption

The strategic risk is bigger than license cost

Locked

Data & workflows

Opaque formats and vendor-specific automation make change expensive.

Fragmented

Engineering flow

Design intent is handed between disconnected capture, simulation, and physical tools.

Constrained

Innovation

New methods wait for vendor roadmaps, scarce specialists, or brittle integrations.

> The question is not “Can we buy another tool?” It is “Can we preserve strategic control of how we design?”

The proposal: add optionality, not disruption

Start beside the incumbent flow

- Select a **bounded, non-tapeout-critical** design
- Connect one relevant **PDK and simulator path**
- Reproduce a known result
- Measure engineering effort and workflow fit
- Keep every pilot artifact portable

Earn expansion with evidence

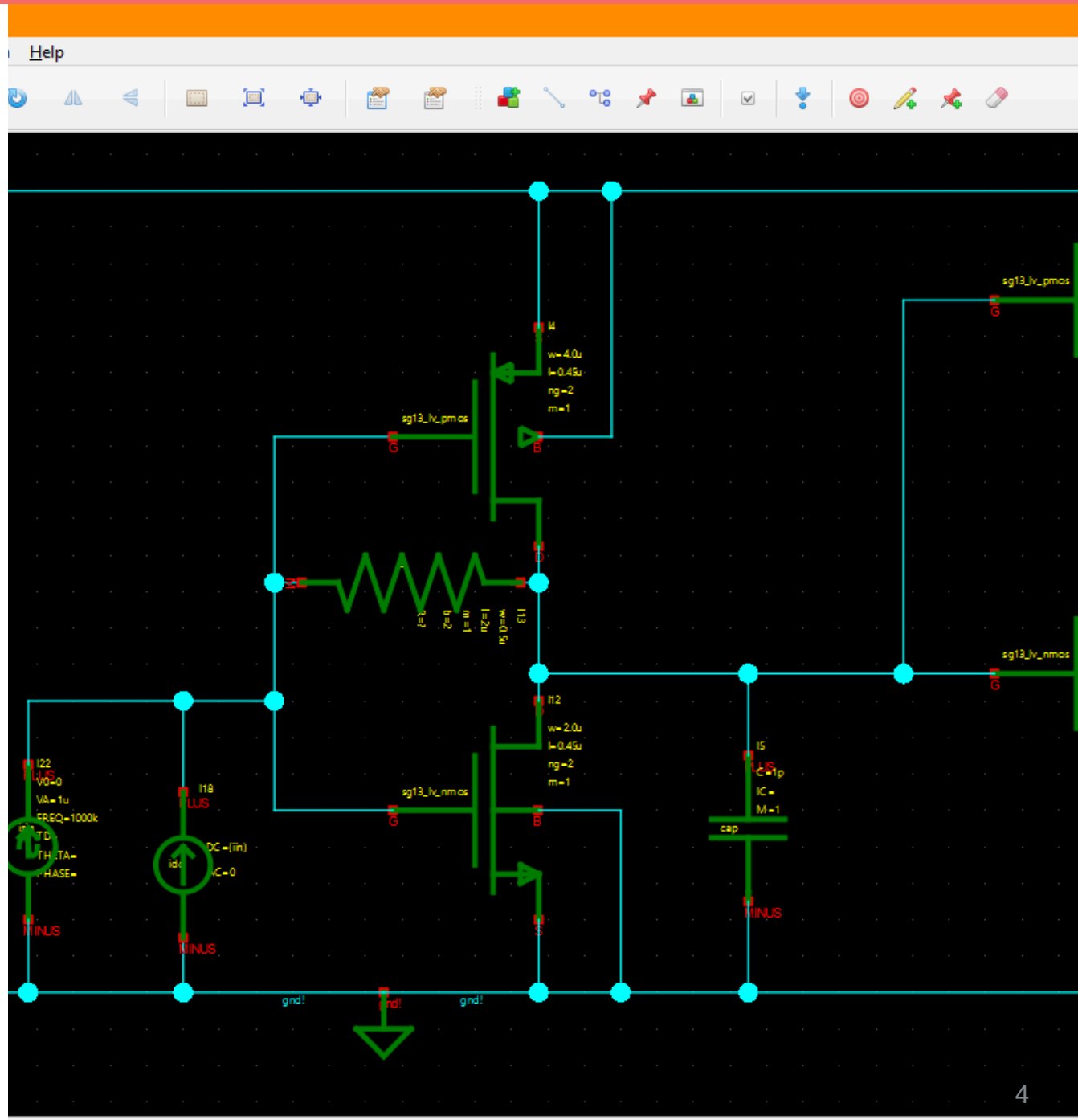
No forced migration
No proprietary database commitment
No enterprise-wide rollout to approve

A pilot can answer the real question:

> Where does Revolution EDA create leverage for us now?

One environment across the custom IC loop

- **Schematic & symbol** editing with typed parameters and hierarchy navigation
- Hierarchical, configuration-driven **multi-simulator netlisting**
- Optional **simulation, plotting, LVS, and AI** plugins
- Connectivity-aware **layout** with Python PCells and SDL integration
- **EDIF 2 0 0, GDS, and OAS** interoperability paths
- RC parasitic extraction to **SPICE, SPEF, Spectre, and VACASK**



Version 1.0.0: a broader, more integrated platform

Design

Production-shaped editing

Typed instance parameters, net expressions and inherited connections, hierarchy navigation, persistent check markers, find/replace, arrays, and unified copy/paste workflows.

Flow

Connected verification

One connectivity model now serves editing, diagnostics, netlisting, probing, SDL, LVS integration, and RC extraction across schematic and layout.

Deploy

Managed extensibility

Plugin registry, compatibility and capability negotiation, update discovery, commercial licensing, and standalone Windows/Linux distribution paths.

> This is a release milestone—not a blanket signoff qualification. Readiness remains PDK-, plugin-, and flow-specific.

Design data your organization can inspect

Human-readable JSON

- Diffable in standard version-control systems
- Scriptable with familiar engineering tools
- Reviewable without a proprietary database reader
- Suitable for automation and reproducible transformations

```
{
  "cellView": "schematic",
  "viewType": "schematic"
}
```

> The design database becomes an accessible engineering asset—not a black box.

Integrate where standards already exist

Boundary	Revolution EDA path	Executive value
Process enablement	Python PDK modules, callbacks, and PCells	Adapt the environment to the process
Simulation	Xyce, Spectre, and VACASK netlisting paths	Preserve simulator choice
Design intake	EDIF 2 0 0, Verilog-A, Spectre and VACASK subcircuits	Reuse established design and model assets
Physical exchange	GDS / OAS import and export	Fit established downstream flows
Verification	Hierarchical LVS integration and RC extraction	Connect intent, comparison, and analysis
Extracted output	SPICE / SPEF / Spectre / VACASK	Feed existing downstream stages
Automation	Python console, JSON, versioned plugin capabilities	Build workflow IP with common skills

A platform, not a fixed appliance

PDK layer

Process layers, callbacks, labels, and PCells are supplied through Python packages.

Plugin layer

Plugins add menus, custom views, simulation, plotting, LVS, SDL, and AI capabilities without changing core code.

Automation layer

An integrated Python console and transparent files expose practical extension points.

Build differentiated workflow IP at your pace

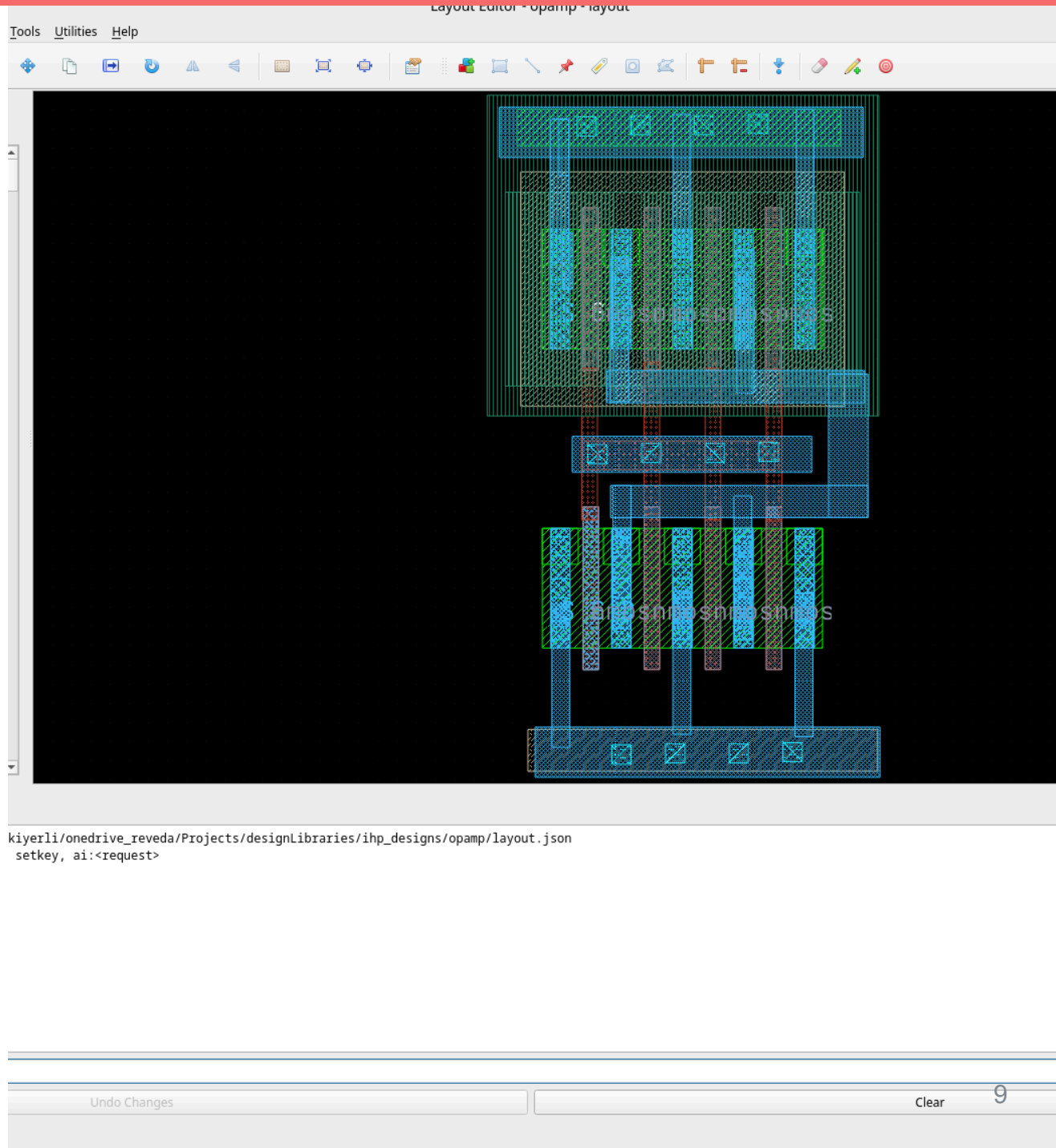
Source and binary plugin models allow **open collaboration** and **controlled commercial extensions** to coexist. Version/capability checks and in-app update discovery make that ecosystem manageable.

AI assistance—with a reversible operating model

- Natural-language editing of **schematics, symbols, and layouts**
- Choice of **OpenAI, Google, or Anthropic** backends
- A backup before each AI modification
- One-click restore / undo path
- Direct inspection of design JSON
- Encrypted local storage for API keys

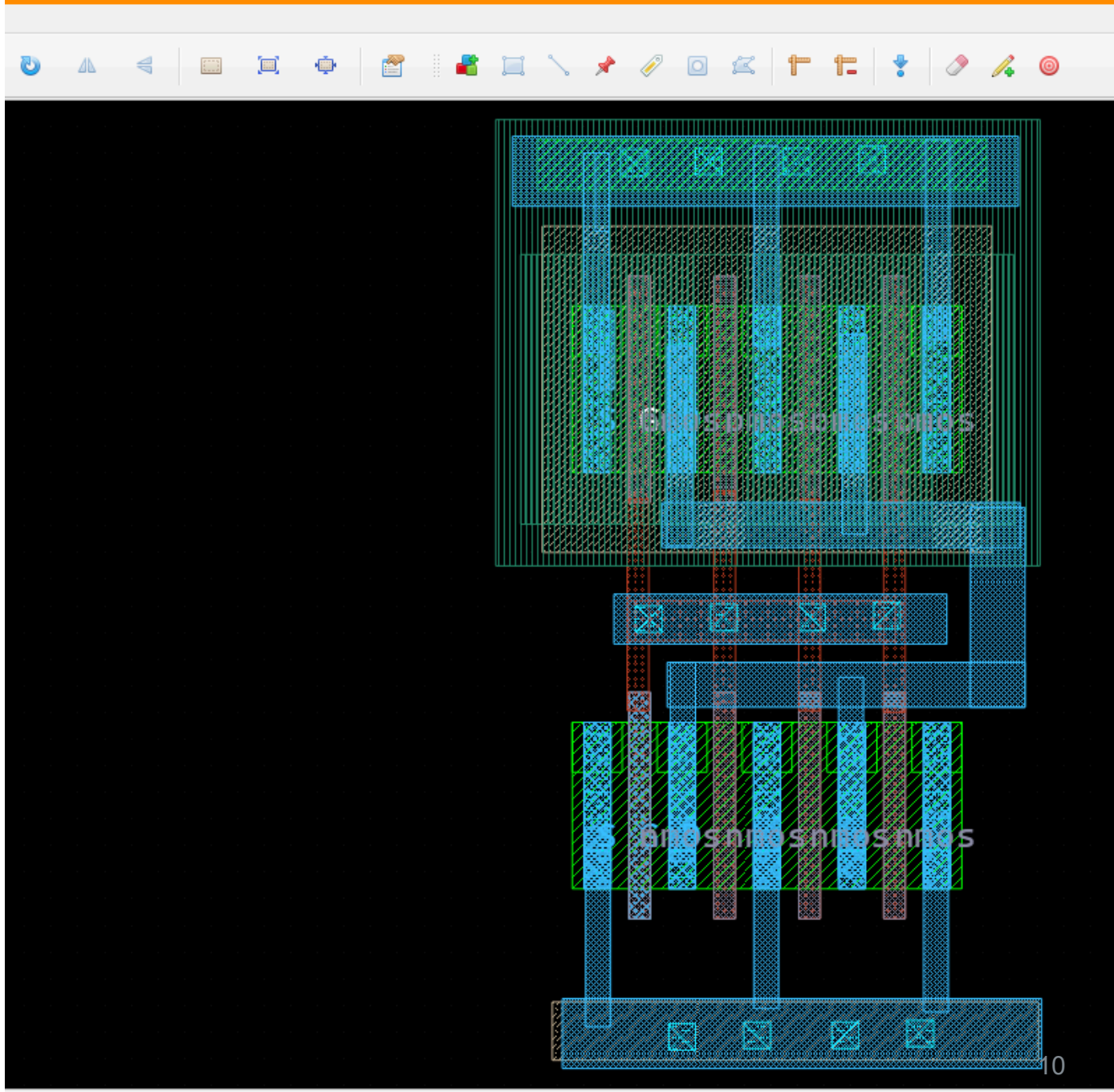
Treat AI as an accelerator—not an authority

Deployment should follow company policy for model access, design-data handling, review, and approval.



Physical design remains connected to intent

- Hierarchical custom layout with configurable display depth
- Rectangles, paths, polygons, pins, labels, and via arrays
- Python parametric cells with editable parameters
- Layer visibility, selectability, and used-layer filtering
- Versioned schematic-driven-layout adapter for compatible plugins
- Shared connectivity, net naming, probing, and highlighting



The business case is strategic leverage

For design leadership

Another path for prototyping, education, research, smaller programs, and workflow experiments.

For engineering

One inspectable environment with Python automation and fewer hard boundaries between design stages.

For the enterprise

Reduced dependence on any single interface, file format, deployment model, or vendor roadmap.

> Optionality compounds: every reusable PDK element, plugin, and automation script increases future choice.

Open core changes the control model

MPL 2.0 licensed core

- Use for any purpose
- Inspect and modify source
- Distribute under the license terms
- Build commercial services and derivatives

Practical implications

- Audit the implementation
- Preserve access to the core
- Qualify changes internally
- Keep plugins, PDKs, and libraries under their own applicable terms

This slide is an operational summary, not legal advice. Legal review should confirm obligations for the organization’s intended distribution model.

Address the adoption risks directly

Leadership concern	Pilot control
“Is it production mature?”	Treat v1.0.0 as an evaluation milestone; qualify the exact PDK, plugins, and signoff handoffs
“Will it disturb the current flow?”	Run side-by-side; exchange only through agreed files and interfaces
“Can it support our process?”	Make one PDK plus DRC/LVS/simulator path a pass/fail workstream
“Can extensions be governed?”	Pin compatible versions, review capabilities, and test registry updates before rollout
“What about security and IP?”	Apply approved AI settings, dependency review, access controls, and artifact handling
“Will it scale?”	Test a representative hierarchy and dataset; record load, edit, netlist, check, and export behavior

A focused pilot can produce a decision

Suggested scope: one representative block, one PDK, one simulator path

1 — Establish

Install v1.0.0, register the PDK and libraries, import via EDIF or recreate the reference design, and capture baseline effort.

2 — Exercise

Edit hierarchy, netlist and simulate, exercise DRC/LVS handoffs, change layout, exchange GDS/OAS, and test one automation.

3 — Decide

Compare results, document gaps, estimate ownership needs, and choose stop / extend / adopt.

Deliverables: reproducible project • verified result comparison • issue log • workflow-fit scorecard • recommendation

Agree on success before the pilot starts

Dimension	Example evidence—not a preclaimed result
Correctness	Connectivity checks, netlist, LVS comparison, and selected simulation outputs meet approved criteria
Interoperability	Required EDIF/model, GDS/OAS, and extracted-netlist paths complete without material data loss
Productivity	Engineers complete defined edits and automation tasks; effort is recorded against baseline
Extensibility	One small plugin, script, PCell, or PDK adaptation is implemented and reviewed
Governance	Project is reproducible from controlled source, configuration, and dependency records
Adoption	Pilot users identify credible use cases and blocking gaps

Score the evidence. Do not grade the demo.

Authorize a controlled evaluation

Choose one sponsor, one engineering lead, one representative design, and one explicit decision date.

The ask

Approve the pilot scope and success criteria—not a migration.

The downside is bounded.
The learning is reusable.
The upside is strategic design freedom.